

Relatório Técnico: Sistema de Gerenciamento de Estoque

Autor: Cleyton Dn

Data: 30 de junho de 2026

Versão: 1.0

Objetivo do Projeto

O **Sistema de Gerenciamento de Estoque** foi desenvolvido com o objetivo de demonstrar a aplicação dos conceitos de **Programação Orientada a Objetos (POO)** em Python. O sistema permite o **cadastro, atualização e consulta de produtos em um estoque**, além de cálculos como o **valor total do estoque** e a **identificação de produtos por preço**.

Escopo

O sistema abrange as seguintes funcionalidades:

- Cadastro de produtos (nome, quantidade, preço).
- Atualização do estoque (venda ou reposição).
- Cálculo do valor total do estoque.
- Identificação do produto mais caro e mais barato.
- Listagem de todos os produtos cadastrados.

Público-Alvo

- Estudantes de programação que desejam entender a **aplicação prática de POO**.
- Pequenos comerciantes que precisam de um **sistema simples para controle de estoque**.
- Professores que buscam **exemplos didáticos** para aulas de POO.

1. Visão Geral

Sistema: Sistema de Gerenciamento de Estoque

Linguagem: Python 3.x

Objetivo: Gerenciar produtos, quantidades e preços em um estoque, com interface via terminal.

2. Estrutura do Código

2.1 Classes e Hierarquia

Produto (Classe Base)

└─ ProdutoPerecivel (Classe Derivada)

Estoque (Classe de Gerenciamento)

2.2 Classe Produto

Responsabilidade: Representar um produto genérico.

Atributos:

Atributo	Tipo	Descrição
_nome	str	Nome do produto
_quantidade	int	Quantidade em estoque (≥ 0)
_preco	float	Preço unitário (≥ 0)

Métodos:

- `__init__(nome, quantidade, preco)`: Construtor
- `__repr__()`: Representação do objeto
- **Properties**: nome, quantidade, preco com validação
- **Setters**: Validações de tipo e valor

2.3 Classe ProdutoPecivel

Responsabilidade: Especialização de produto com validade.

Atributos Adicionais:

Atributo	Tipo	Descrição
_data_validade	str	Data de validade (string não vazia)

Diferenças:

- Herda de Produto
- Adiciona propriedade `data_validade` com validação
- Sobrescreve `__init__` para incluir data de validade

2.4 Classe Estoque

Responsabilidade: Gerenciar coleção de produtos.

Atributos:

Atributo	Tipo	Descrição
produtos	List[Produto]	Lista de produtos

Métodos Principais:

Método	Parâmetros	Retorno	Descrição
<code>cadastrar_produto</code>	nome, quantidade, preco	None	Adiciona novo produto, evita duplicatas

atualizar_estoque	nome, quantidade, operacao	None	Venda/reposição com validação de saldo
calcular_valor_total_estoque	-	float	Soma dos valores (qtd × preço)
identificar_produto_mais_caro	-	Produto/None	Produto com maior preço
identificar_produto_mais_barato	-	Produto/None	Produto com menor preço
listar_produtos	-	None	Exibe todos os produtos com valores
mostrar_valor_individual	nome	None	Exibe preço de um produto específico

3. Funcionalidades

3.1 Menu Interativo

- Cadastrar produto
- Atualizar estoque (venda/reposição)
- Calcular valor total
- Mostrar valor individual
- Identificar produto mais barato
- Listar todos os produtos

3.2 Validações

- **Produto:** Nome não vazio, quantidade ≥ 0 , preço ≥ 0
- **Operações:** Quantidade não negativa, saldo suficiente para venda
- **Entrada:** Tratamento de exceções para inputs inválidos

4. Pontos Fortes

1. **Encapsulamento:** Uso de properties para controle de atributos
2. **Validação robusta:** Verificações de tipo e valor
3. **Tratamento de exceções:** Try/except para entradas inválidas
4. **Case-insensitive:** Busca de produtos ignorando maiúsculas/minúsculas
5. **Herança:** Extensão para ProdutoPerecível possível
6. **Prevenção de duplicatas:** Validação no cadastro

5. Recomendações de Melhoria

5.1 Alto Impacto

1. Persistência de dados

- a. Implementar salvamento em arquivo (JSON/CSV)
- b. Carregar dados na inicialização

2. ProdutoPercível não utilizado

- a. Integrar no menu com opção de cadastro
- b. Adicionar verificação de validade expirada

3. Mensagens de erro inconsistentes

- a. Padronizar saídas (ex: sempre usar `print()`)
- b. Criar constante para mensagens

5.2 Médio Impacto

4. Validação de preço

- a. Adicionar limite máximo/realista
- b. Permitir formatação com vírgula

5. Performance

- a. Usar dicionário para produtos ($O(1)$ busca)
- b. Atualizar `identificar_produto_mais_caro/barato` com `min/max`

6. Documentação

- a. Adicionar docstrings em todos os métodos
- b. Comentários explicativos para partes complexas

5.3 Baixo Impacto

7. Interface

- a. Tabelas formatadas para listagem
- b. Cores para destacar mensagens

8. Testes

- a. Criar suite de testes unitários (unittest/pytest)
- b. Testar casos de borda

6. Métricas

6.1 Complexidade

- **Classes:** 3
- **Métodos:** 14
- **Linhas de código:** ~150
- **Ciclo de vida:** Simples

6.2 Cobertura de Funcionalidades

Funcionalidade	Implementada
CRUD produtos	✅ Parcial
Validações	✅ Completa

Cálculos	✓ Completa
Interface	✓ Terminal

7. Exemplos de Uso

7.1 Cadastro de Produto

```
>>> estoque.cadastrar_produto("Café", 10, 25.99)
Produto 'Café' cadastrado com sucesso!
```

7.2 Operação de Venda

```
>>> estoque.atualizar_estoque("Café", 3, "venda")
Estoque de 'Café' atualizado. Nova quantidade: 7
```

7.3 Cálculo de Valor Total

```
>>> estoque.calcular_valor_total_estoque()
R$ 181.93
```

8. Conclusão

O sistema apresenta uma base sólida para gerenciamento de estoque com validações robustas e estrutura orientada a objetos bem definida. A arquitetura permite extensões como persistência e novos tipos de produtos. Recomenda-se implementar as melhorias propostas para torná-lo um sistema completo e profissional.